

# TrackShield

Browser Extension to Block Trackers

|          |                                     |
|----------|-------------------------------------|
| Student: | Jiya Daulat                         |
| Domain:  | Web Development / Browser Extension |
| Tools:   | JavaScript, Manifest V3, HTML/CSS   |
| Date:    | June 2026                           |

---

## 1. Introduction

Online trackers are pieces of code embedded in websites that monitor user behavior, collect browsing data, and build detailed profiles for advertising purposes. Every time you visit a website, dozens of third-party scripts from companies like Google, Facebook, and advertising networks silently execute in your browser. TrackShield is a Chrome extension built using Manifest V3 that intercepts and blocks these tracking scripts in real time, giving users full visibility and control over their privacy.

## 2. Abstract

TrackShield is a privacy-focused Chrome browser extension developed using JavaScript, HTML/CSS, and Chrome's Manifest V3 APIs. It maintains a list of 40 known tracking domains, intercepts network requests using the **declarativeNetRequest** API, and additionally monitors DOM mutations via a content script to catch dynamically injected trackers. Blocked counts are displayed as a real-time badge on the extension icon. Users can whitelist trusted sites, add custom domains to a block list, and view lifetime analytics through a settings panel.

## 3. Tools & Technologies Used

| Technology                | Purpose                                                         |
|---------------------------|-----------------------------------------------------------------|
| JavaScript (ES6+)         | Core extension logic, background service worker, content script |
| Chrome Manifest V3        | Extension configuration, permissions, lifecycle management      |
| declarativeNetRequest API | Network-level blocking of tracker domains                       |
| MutationObserver API      | DOM-level interception of dynamically injected scripts          |
| chrome.storage.local      | Persisting whitelist, blacklist, and statistics                 |
| chrome.webNavigation API  | Detecting page loads to reset per-tab counters                  |
| HTML5 / CSS3              | Popup UI and options settings page                              |
| Python + Pillow           | Icon generation (development only)                              |

## 4. Steps Involved in Building the Project

### Step 1 - Project Setup and manifest.json

Created the extension folder structure with pages/ and icons/ subdirectories. Wrote manifest.json using Manifest V3, declaring permissions (declarativeNetRequest, storage, tabs, webNavigation), linking the background service worker, content script, and popup page.

### **Step 2 - Tracker Rules Database (tracker\_rules.json)**

Built a JSON file with 40 declarativeNetRequest rules covering major trackers: Google Analytics, Google Tag Manager, Facebook Pixel, Hotjar, Mixpanel, Segment, Amplitude, FullStory, Heap, Criteo, Taboola, Outbrain, Amazon Ads, and more. Each rule specifies the URL filter and which resource types to block.

### **Step 3 - Background Service Worker (background.js)**

Implemented the persistent background script that listens for webNavigation.onCommitted to reset per-tab counters, receives TRACKER\_BLOCKED messages from the content script, maintains in-memory stats by tab ID, updates the badge counter, and responds to GET\_STATS requests from the popup.

### **Step 4 - Content Script (content.js)**

Wrote a content script injected at document\_start on all URLs. It overrides document.createElement to intercept dynamically created script tags and suppress tracker sources before they load. A MutationObserver watches for script/image nodes added to the DOM and removes known trackers.

### **Step 5 - Popup UI (popup.html / popup.js)**

Built the extension popup showing a live blocked count for the current page, a list of blocked tracker domains, a pause/resume toggle, a one-click whitelist button for the current site, and a lifetime total counter.

### **Step 6 - Options/Settings Page (options.html / options.js)**

Created a full settings page where users can add/remove sites from the whitelist and add custom domains to a personal blacklist. An analytics dashboard shows total blocked count, whitelist size, and custom block rules.

### **Step 7 - Icons and Packaging**

Generated 16x16, 48x48, and 128x128 shield icons using Python and Pillow. Packaged all files into a ZIP archive for loading as an unpacked extension in Chrome.

## **5. Conclusion**

TrackShield demonstrates how browser extension APIs can be used to meaningfully improve user privacy with minimal overhead. The use of Chrome's declarativeNetRequest API ensures network-level blocking happens efficiently without requiring a persistent background page to inspect every request. The dual-layer approach combining declarative rules with a MutationObserver content script catches both static and dynamically injected trackers. The extension is designed to be maintainable: the tracker domain list can be updated independently, and users retain full control via the whitelist/blacklist system. This project provides hands-on experience with Manifest V3 migration patterns, service worker lifecycle, cross-context messaging, and Chrome storage APIs.